

Wyszukiwanie binarne

Dane: $A[0..n-1]$,
 $A[0] \leq A[1] \leq \dots \leq A[n-1]$,
 X

Wynik: dowolne i takie, że $A[i]=X$, jeśli istnieje;
-1 jeśli takie i nie istnieje.

Wersja klasyczna ("dwa porównania")

```
L := 0; R := n-1; found := false;
while R-L >= 0 and not found do
  m = ⌊(L+R)/2⌋;
  if X = A[m] then found := true;
  else
    if X < A[m] then R := m-1;
    else L := m+1;

if found then
  return m;
else
  return -1;
```

Poprawność:

Niezmiennik: jeśli X jest w $A[0..n-1]$, to X jest w $A[L..R]$

Złożoność: $\Theta(\log n)$

bo wielkość tablicy w kolejnych iteracjach maleje dwukrotnie.

Wada: mało elastyczny w innych zastosowaniach.

Wersja "jedno porównanie"

Algorytm L

zwraca pewien indeks z zakresu $0..n-1$.

```

L := 0;   R := n-1;
while L < R do
    // jak długo przedział co najmniej 2-elementowy
    m := ⌊(L+R)/2⌋;
    if X ≤ A[m] then
        R := m;
    else
        L := m+1;
        // teraz L=R,
        // X występuje w A ⇔ A[L]=X
return L;
        // na koniec sprawdź czy X=A[L];
end.

```

Przykład:

X=4

indeksy:	0	1	2	3	4	5	6
wartości:	2	4	4	4	4	6	8
	L						R
				m			
	L			R			
		m					
	L	R					
	m						
		LR					

Fakt:

Jeśli X występuje w A , to algorytm L znajduje jego pierwsze wystąpienie od lewej - czyli najmniejsze i takie że $A[i]=X$.

Jaki jest wynik algorytmu jeśli X nie występuje w A ?

Wykonaj test dla: $X=7$, $X=1$, $X=9$

indeksy: 0 1 2 3 4 5 6

wartości: 2 4 4 4 4 6 8

(przelicz) - - - - -

Fakt:

Jeśli X nie występuje w A , oraz $X < A[n-1]$, to algorytm znajduje pierwsze wystąpienie najmniejszego elementu większego niż X .

Jeśli $X > A[n-1]$, to algorytm zwraca $n-1$, tę samą wartość co w przypadku $A[n-2] < X < A[n-1]$.

Uogólnijmy algorytm aby jednolicie obsługiwał też przypadek $X > A[n-1]$.

Obserwacja:

Dzięki temu, że w wyrażeniu obliczającym wartość m posługujemy się podłogą, w pętli nigdy nie porównujemy X z $A[n-1]$.

Można zatem hipotetycznie założyć, że ostatnim elementem jest wartownik $A[n]=+\infty$, nie przechowując tego elementu. Otrzymujemy:

Algorytm Lowerbound

```
L := 0; R := n;           // zamiast R := n-1
while L < R do
    m :=  $\lfloor (L+R)/2 \rfloor$ ;
    if X <= A[m] then
        R := m;
    else
        L := m+1;
return L.
```

Fakt:

Algorytm Lowerbound zwraca pozycję pierwszego wystąpienia elementu większego lub równego X, n jeśli $X > A[n-1]$.

Inaczej: pierwszą od lewej pozycję, na którą możemy wstawić X (rozsuwając elementy w tablicy od tego miejsca w prawo), zachowując porządek w tablicy.

Symetrycznie, możemy znajdować ostatnie wystąpienie szukanego elementu. Należy posłużyć się operacją "sufit" – uzasadnienie łatwo zauważyć dla tablicy dwuelementowej o takich samych wartościach – gdy użyjemy "podłogi" procedura się pętli.

Elegantsze rozwiązanie polega na wyszukiwaniu pozycji pierwszej za ostatnim wystąpieniem danej wartości, za pomocą niewielkiej modyfikacji algorytmu Lowerbound.

Żądany rezultat uzyskamy, jeśli szukać będziemy wartości większej niż X , ale mniejszej niż następna po X wartość w tablicy A ("o epsilon większej niż X ").

Wystarczy, że w pętli sytuacji $X=A[m]$ traktować będziemy tak jak przypadek $X>A[m]$ (tzn. pójdziemy wtedy w prawo).

Czyli wystarczy zamienić " $X \leq A[m]$ " na " $X < A[m]$ ".

Algorytm Upperbound

```
L := 0; R := n;
while L < R do
    m :=  $\lfloor (L+R)/2 \rfloor$ ;
    if  $X < A[m]$  then
        R := m;
    else
        L := m+1;
return L.
```

Fakt:

Algorytm Upperbound znajduje pozycję pierwszego elementu większego niż X w tablicy A .

Inaczej: ostatnią pozycję, na którą możemy wstawić X (rozsuwając elementy w tablicy od tego miejsca w prawo), zachowując porządek w tablicy.

Fakt:

Jeśli X nie występuje w tablicy A , to wyniki Lowerbound i Upperbound są identyczne.

Obserwacja:

Dla tablicy pustej mamy $n=0$. Oba algorytmy zwracają $L=0$, czyli poprawny wynik.

Zastosowanie 1:

Problem: Podaj liczbę wystąpień X w $A[0..n-1]$.

Rozwiązanie:

$$\text{Upperbound}(A,X) - \text{Lowerbound}(A,X).$$

Złożoność: $\Theta(\log n)$.

Zauważ, że naiwne podejście:

- znajdź binarnie dowolne wystąpienie X w $A[]$,
- przeglądaj w lewo i prawo, aby policzyć ile jest wystąpień X ,

ma złożoność pesymistyczną liniową.

Zastosowanie 2 - wyszukiwanie "po wyniku":

Rozwiąż równanie $W(x)=x^3 - 41x^2 - 34x - 336=0$.
Należy podać sufit z pierwiastka p tego wielomianu.

Założenia, znane skądinąd:

- $-100 < p < 1000$,
- $W(-100) < 0$,
- $W(1000) > 0$,
- w przedziale $[-100, 1000]$ funkcja rosnąca.

Rozwiązanie: stosujemy dowolny z przedstawionych algorytmów wyszukiwania binarnego, np. algorytm Upperbound:

```
L := -100 ; R := 1000;
while L < R do
  m := ⌊ (L+R) / 2 ⌋ ;
  // wielomian obliczamy metodą Hornera
  if m*(m*(m-41)-34)-336 > 0 then
    R := m;
  else
    L := m+1;
return L.
```

Jest to wariant metody bisekcji znajdowania pierwiastka funkcji - tutaj znajdujemy przybliżenie tego pierwiastka będące liczbą całkowitą.